

Modulo 2A — Gestione Utenti e Permessi

Corso: Lab Amministrazione di Sistemi T

Prerequisito: Moduli 0A, 0B, 1A, 1B completati

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Contesto

In Modulo 0A hai letto la stringa di permessi `drwxr-xr-x` e ne hai imparato la struttura. In questo modulo la padroneggi operativamente: crei utenti reali, assegna permessi, testi gli accessi effettivi con utenti diversi. È il modulo con la connessione più diretta a Security: **privilege escalation** — la tecnica con cui un attaccante passa da utente normale a root — si fonda quasi sempre su una misconfigurazione dei permessi o di `sudo`.

Crea la directory di lavoro:

```
mkdir ~/Lab2A && cd ~/Lab2A
```

Parte 1 — Il modello utenti di Linux

1.1 Come Linux identifica gli utenti

Linux non conosce i nomi: internamente tutto è un numero. Ogni utente ha uno **UID** (User ID), ogni gruppo ha un **GID** (Group ID). Il nome (`vagrant`, `root`) è solo una traduzione leggibile memorizzata in `/etc/passwd`.

```
id
cat /etc/passwd | grep vagrant
cat /etc/passwd | grep root
```

Struttura di una riga di `/etc/passwd`:

```
vagrant:x:1000:1000:,,,:/home/vagrant:/bin/bash
|      | |      |      |      |
nome   | UID   GID   GECOS  home      shell
```

password (sempre "x" - la vera hash è in `/etc/shadow`)

I valori UID hanno una convenzione: - 0 — root (sempre) - 1-999 — utenti di sistema (servizi, demoni) - 1000+ — utenti umani reali

Verificare il proprio UID/GID

```
id
```

Vedere tutti gli utenti del sistema

```
cat /etc/passwd
```

Vedere solo i gruppi dell'utente corrente

```
groups
```

1.2 Il file /etc/shadow

Le password non sono in /etc/passwd (leggibile da tutti) ma in /etc/shadow, accessibile solo da root. Contiene gli hash delle password e le politiche di scadenza.

```
sudo cat /etc/shadow | grep vagrant
```

Struttura:

```
vagrant:$6$abc123...:19800:0:99999:7:::  
|       |       | | | | |  
nome    hash pw   gg ult  min max warn
```

Parte 2 — Gestione degli utenti

2.1 Creare un utente — useradd

```
# Crea l'utente "alice" con home directory  
sudo useradd -m -s /bin/bash alice  
  
# Verifica che sia stato creato  
id alice  
cat /etc/passwd | grep alice  
ls /home/
```

Il flag `-m` crea la home directory `/home/alice`. Il flag `-s /bin/bash` assegna Bash come shell di login. Senza `-m` non viene creata la home; senza `-s` viene assegnata una shell di default (spesso `sh` o nessuna).

```
# Imposta la password per alice  
sudo passwd alice  
# (inserisci una password quando richiesto, es. "alice123")
```

2.2 Modificare un utente — usermod

```
# Creare un secondo utente  
sudo useradd -m -s /bin/bash bob  
  
# Aggiungere bob al gruppo "sudo" (gli dà i privilegi di amministratore)  
sudo usermod -aG sudo bob  
  
# Verificare i gruppi di bob  
groups bob  
id bob
```

Il flag `-aG` significa **append to Group** — aggiunge bob al gruppo `sudo` senza rimuoverlo dai gruppi esistenti. Senza `-a`, `-G` sostituisce tutti i gruppi con quello specificato — errore comune e pericoloso.

2.3 Cambiare utente — su

```
# Passare all'utente alice (richiede la password di alice)
su - alice

# Verificare chi si è adesso
whoami
id
pwd

# Tornare a vagrant
exit
```

Il trattino in `su - alice` è importante: carica l'ambiente completo di alice (variabili d'ambiente, directory di lavoro, ecc.). `su alice` senza trattino cambia utente ma mantiene l'ambiente del chiamante — comportamento spesso indesiderato.

2.4 Eliminare un utente — userdel

```
# Elimina l'utente senza cancellare la home
sudo userdel alice

# Elimina l'utente E la sua home directory
sudo userdel -r alice
```

Verificare che alice non esista più:

```
id alice    # deve dare: no such user
ls /home/   # /home/alice non c'è più (se usato -r)
```

Parte 3 — I permessi in dettaglio

3.1 Riepilogo della stringa di permessi

Già vista in Modulo 0A, ma adesso la usi operativamente. Ogni file ha una stringa di 10 caratteri:

```
-  r w x  r - x  r - -
|  |||||  |||||  |||||
|  owner  group  altri
|
```

tipo: - = file, d = directory, l = link simbolico

I tre blocchi `rwX` si ripetono per tre categorie: - **owner** (proprietario del file) - **group** (gruppo proprietario) - **others** (tutti gli altri)

Ogni lettera vale un numero in notazione ottale: - `r` = 4 - `w` = 2 - `x` = 1

I tre valori si sommano per ogni blocco: `rwX` = 4+2+1 = **7**, `r-x` = 4+0+1 = **5**, `r--` = 4+0+0 = **4**.

Esempi di notazione ottale: | Stringa | Ottale | Significato pratico | `|---|---|` | `rwXr-Xr-X` | 755 |
Default directory — owner tutto, gli altri leggono ed eseguono | `|rw-r--r--` | 644 | Default file —
owner legge/scrive, gli altri solo leggono | `|rwX-----` | 700 | Solo l'owner può accedere | `|rw-----`

| 600 | Solo l'owner legge/scrive, nessun altro nulla | | `rw-rw-rw-r` | 777 | Tutti possono fare tutto — pericoloso |

3.2 Modificare i permessi — `chmod`

```
# Crea un file di test
echo "contenuto segreto" > file_test.txt
ls -la file_test.txt
# output: -rw-r--r-- 1 vagrant vagrant ...
```

Notazione ottale (la più usata):

```
chmod 600 file_test.txt    # solo owner legge/scrive
chmod 644 file_test.txt    # default file
chmod 755 file_test.txt    # default directory o script
chmod 700 file_test.txt    # solo owner, nessun altro
chmod 000 file_test.txt    # nessuno può fare nulla (neanche leggere)
```

Notazione simbolica (più leggibile per modifiche puntuali):

```
chmod u+x file_test.txt    # aggiunge esecuzione all'owner (u = user/owner)
chmod g-w file_test.txt    # rimuove scrittura al gruppo (g = group)
chmod o-r file_test.txt    # rimuove lettura agli altri (o = others)
chmod a+r file_test.txt    # aggiunge lettura a tutti (a = all)
```

Esercizio pratico — testare il blocco dei permessi:

```
chmod 000 file_test.txt
cat file_test.txt          # → Permission denied
chmod 644 file_test.txt
cat file_test.txt          # → "contenuto segreto" (funziona)
```

3.3 Modificare il proprietario — `chown` e `chgrp`

```
# Ri-creare l'utente alice per questo esercizio
sudo useradd -m -s /bin/bash alice
sudo passwd alice
```

```
# Creare un file come vagrant
echo "file di vagrant" > file_vagrant.txt
ls -la file_vagrant.txt
# output: -rw-r--r-- 1 vagrant vagrant ...
```

```
# Cambiare il proprietario ad alice
sudo chown alice file_vagrant.txt
ls -la file_vagrant.txt
# output: -rw-r--r-- 1 alice vagrant ...
```

```
# Cambiare sia proprietario che gruppo in un solo comando
sudo chown alice:alice file_vagrant.txt
ls -la file_vagrant.txt
```

```
# output: -rw-r--r-- 1 alice alice ...
```

```
# Cambiare solo il gruppo
```

```
sudo chgrp vagrant file_vagrant.txt
```

```
ls -la file_vagrant.txt
```

```
# output: -rw-r--r-- 1 alice vagrant ...
```

chown richiede sudo perché solo root può cambiare il proprietario di un file. Un utente può cambiare il gruppo solo verso gruppi di cui fa parte.

Parte 4 — Privilegi elevati: sudo

4.1 Come funziona sudo

sudo (superuser do) permette a un utente autorizzato di eseguire un singolo comando come root, senza diventare root permanentemente. La configurazione che stabilisce chi può usare sudo e per quali comandi sta in `/etc/sudoers`.

```
# Vedere il contenuto di /etc/sudoers (non modificarlo mai direttamente)
```

```
sudo cat /etc/sudoers
```

La riga chiave per Debian/Ubuntu è:

```
%sudo    ALL=(ALL:ALL) ALL
```

Traduzione: il gruppo sudo (%sudo) può eseguire qualsiasi comando (ALL) su qualsiasi host, come qualsiasi utente. È la configurazione standard.

```
# Verificare se vagrant è nel gruppo sudo
```

```
groups vagrant
```

```
# output deve includere: sudo
```

```
# Eseguire un comando come root senza diventare root
```

```
sudo whoami          # → root
```

```
whoami               # → vagrant (si è ancora vagrant)
```

4.2 sudo -l — vedere cosa si può fare

```
sudo -l
```

Questo comando elenca tutti i comandi che l'utente corrente può eseguire con sudo. È uno dei primi comandi che un attaccante esegue dopo aver ottenuto accesso a una macchina — e per questo è fondamentale saperlo interpretare.

4.3 Aggiungere un utente al gruppo sudo

```
# Bob è già nel gruppo sudo (aggiunto in Parte 2)
```

```
# Verificarlo
```

```
groups bob
```

```
# Testare: passare a bob e usare sudo
```

```
su - bob
sudo whoami
exit
```

4.4 Diventare root — sudo -i e sudo su

```
# Aprire una shell root completa
sudo -i
whoami    # → root
pwd       # → /root

# Uscire dalla shell root
exit
whoami    # → vagrant
```

sudo -i è equivalente a fare login come root. Usare con cautela: da root qualsiasi comando sbagliato viene eseguito senza restrizioni.

Parte 5 — Esercizio integrato

Metti insieme gestione utenti e permessi in uno scenario realistico.

Scenario: creare due utenti (alice e bob), una directory condivisa tra loro, e verificare che i permessi funzionino come previsto.

```
# Step 1 - creare il gruppo condiviso
sudo groupadd progetto

# Step 2 - aggiungere alice e bob al gruppo
sudo usermod -aG progetto alice
sudo usermod -aG progetto bob

# Verificare
groups alice
groups bob

# Step 3 - creare la directory condivisa
sudo mkdir /srv/progetto
sudo chown root:progetto /srv/progetto
sudo chmod 770 /srv/progetto
ls -la /srv/

# Lettura: -rwxrwx--- (770) significa:
# owner (root): rwx - può fare tutto
# group (progetto): rwx - alice e bob possono fare tutto
# others: --- - nessuno degli altri può accedere

# Step 4 - testare come alice
```

```

su - alice
cd /srv/progetto
echo "file di alice" > alice.txt
ls -la
exit

# Step 5 - testare come bob
su - bob
cd /srv/progetto
cat alice.txt          # → funziona (bob è nel gruppo progetto)
echo "file di bob" > bob.txt
exit

# Step 6 - testare come utente fuori dal gruppo
# Creare un terzo utente senza il gruppo
sudo useradd -m -s /bin/bash charlie
sudo passwd charlie
su - charlie
cd /srv/progetto      # → Permission denied (charlie non è nel gruppo progetto)
exit

```

Connessione con Security

Questo modulo è il prerequisito concettuale diretto per **privilege escalation**:

- Un attaccante che ottiene accesso come utente normale (**www-data**, **nobody**, ecc.) cercherà immediatamente di elevare i propri privilegi a root
 - **sudo -l** è il primo comando eseguito: se un utente può eseguire qualcosa con **sudo** senza password, o se può eseguire un interprete (**python**, **vim**, **bash**) con **sudo**, può ottenere una shell root
 - Permessi 777 su file critici, o file di configurazione scrivibili da utenti non privilegiati, sono vettori di escalation classici
 - **/etc/shadow** leggibile da non-root è una vulnerabilità critica (gli hash delle password possono essere craccati offline)
 - Capire la norma — come un sistema ben configurato dovrebbe apparire — è il prerequisito per riconoscere le anomalie che un attaccante sfrutterebbe
-

Riepilogo comandi

Comando	Funzione
<code>useradd -m -s /bin/bash nome</code>	Crea utente con home e shell Bash
<code>passwd nome</code>	Imposta/cambia password utente
<code>usermod -aG gruppo nome</code>	Aggiunge utente a un gruppo (senza rimuoverlo dagli altri)
<code>userdel -r nome</code>	Elimina utente e home directory

Comando	Funzione
<code>groupadd nome</code>	Crea un nuovo gruppo
<code>id [nome]</code>	Mostra UID, GID e gruppi
<code>groups [nome]</code>	Mostra i gruppi dell'utente
<code>su - nome</code>	Cambia utente (con caricamento ambiente)
<code>sudo comando</code>	Esegue un comando come root
<code>sudo -i</code>	Apri shell root completa
<code>sudo -l</code>	Elenca comandi eseguibili con sudo
<code>chmod 755 file</code>	Modifica permessi (notazione ottale)
<code>chmod u+x file</code>	Modifica permessi (notazione simbolica)
<code>chown utente:gruppo file</code>	Cambia proprietario e gruppo
<code>chgrp gruppo file</code>	Cambia solo il gruppo
<code>ls -la</code>	Visualizza permessi di tutti i file

Collegati

- [\[\[Appunti_modulo2A\]\]](#)
- [\[\[appunti_modulo2A_utenti_permessi\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[glossario_sysadm\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[troubleshooting_vm\]\]](#)
 · [\[\[metodo_studio_esami_pratici\]\]](#)